

TCS NQT & Infosys SP/DSE Coding Questions

Shift-wise Problem Statements, Constraints, Edge Cases, and Python Solutions

1. TCS NQT — 6th July (Shift 2)

Problem 1: N-th Term of Fibonacci Series

Problem Statement

Given an integer N, write a program to find and output the N-th number in the Fibonacci sequence. The sequence starts with $F(1) = 0$, $F(2) = 1$, and subsequent terms are defined as $F(N) = F(N-1) + F(N-2)$.

Constraints

- $1 \leq N \leq 90$
- Time Limit: 1.0s, Memory Limit: 256MB

Edge Cases Handled

- $N = 1$ and $N = 2$ return base values 0 and 1 directly.
- Space-optimized iterative approach avoids stack overflow for large N.

Python Solution

```
def solve_fibonacci_nth(n: int) -> int:
    if n == 1:
        return 0
    if n == 2:
        return 1

    prev2, prev1 = 0, 1
    current = 0
    for _ in range(3, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return current

if __name__ == "__main__":
    import sys
    data = sys.stdin.read().strip()
    if data:
        print(solve_fibonacci_nth(int(data)))
```

Problem 2: Last Three Terms of an Arithmetic Progression

Problem Statement

Given three space-separated integers representing the first term 'a', common difference 'd', and total terms 'n' of an Arithmetic Progression, print the last three terms: (n-2)-th, (n-1)-th, and n-th terms in order.

Constraints

- $-10^4 \leq a, d \leq 10^4$
- $3 \leq n \leq 10^6$

Edge Cases Handled

- Negative common difference d.
- Minimum length array n = 3.

Python Solution

```
def compute_last_three_ap(a: int, d: int, n: int):
    t_n_minus_2 = a + (n - 3) * d
    t_n_minus_1 = a + (n - 2) * d
    t_n = a + (n - 1) * d
    return t_n_minus_2, t_n_minus_1, t_n

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if len(tokens) >= 3:
        a, d, n = map(int, tokens[:3])
        t1, t2, t3 = compute_last_three_ap(a, d, n)
        print(f"{t1} {t2} {t3}")
```

2. TCS NQT — 9th July (Shift 1)

Problem 3: Find 1-Based Indices of Target in Sorted Array

Problem Statement

Given a sorted array of N integers and a target value X, output all 1-based indices where X appears in the array. If X is not present, return -1.

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{Arr}[i], X \leq 10^9$

Edge Cases Handled

- Target not present in array returns -1.
- Early termination upon encountering elements strictly greater than target.

Python Solution

```
def find_target_indices(arr, target):
    indices = []
    for idx, value in enumerate(arr, start=1):
        if value == target:
            indices.append(idx)
        elif value > target:
            break

    return indices if indices else [-1]

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        n = int(tokens[0])
        arr = [int(x) for x in tokens[1:n+1]]
        target = int(tokens[n+1])
        print(*(find_target_indices(arr, target)))
```

Problem 4: First Unique Element in Array

Problem Statement

Given an array of N integers where exactly one element occurs once and all other elements occur twice, find and output the first unique element using a hash map.

Constraints

- $1 \leq N \leq 10^5$
- $-10^6 \leq \text{Arr}[i] \leq 10^6$

Edge Cases Handled

- Preserves original arrival order when looking up frequency = 1.

Python Solution

```
from collections import Counter
```

```
def find_first_unique_element(arr):
    freq_map = Counter(arr)
    for num in arr:
        if freq_map[num] == 1:
            return num
    return -1

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        n = int(tokens[0])
        arr = [int(x) for x in tokens[1:n+1]]
        print(find_first_unique_element(arr))
```

3. Infosys SP/DSE — 30th July Drive

Problem 5: Constrained Ternary String Generation

Problem Statement

Find the total number of valid strings of length N composed of characters $\{0, 1, 2\}$ such that '00' never appears, '111' never appears, and '2' can only appear in the exact pattern '121'. Output modulo $10^9 + 7$.

Constraints

- $1 \leq N \leq 10^5$

Edge Cases Handled

- Base cases for $N = 1$ and $N = 2$ where '2' cannot exist legally.

Python Solution

```
def count_valid_strings(n: int) -> int:
    MOD = 10**9 + 7
    if n == 1: return 2
    if n == 2: return 3

    end_0, end_1, end_11, end_12 = 1, 1, 0, 0

    for _ in range(2, n + 1):
        next_0 = (end_1 + end_11) % MOD
        next_1 = (end_0 + end_12) % MOD
        next_11 = end_1
        next_12 = end_11
```

```

        end_0, end_1, end_11, end_12 = next_0, next_1, next_11, next_12

    return (end_0 + end_1 + end_11) % MOD

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        print(count_valid_strings(int(tokens[0])))

```

Problem 6: Minimum Partition Cost into K Segments

Problem Statement

Partition an array of N positive integers into K contiguous non-empty subarrays such that the maximum sum among all subarrays is minimized.

Constraints

- $1 \leq K \leq N \leq 10^5$
- $1 \leq \text{Arr}[i] \leq 10^9$

Edge Cases Handled

- $K = 1$ (sum of entire array) and $K = N$ (maximum element).

Python Solution

```

def min_max_subarray_sum(arr, k):
    def can_partition(max_sum):
        segments, curr = 1, 0
        for num in arr:
            if curr + num > max_sum:
                segments += 1
                curr = num
            else:
                curr += num
        return segments <= k

    low, high = max(arr), sum(arr)
    ans = high
    while low <= high:
        mid = (low + high) // 2
        if can_partition(mid):
            ans = mid
            high = mid - 1
        else:

```

```

        low = mid + 1
    return ans

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        n, k = int(tokens[0]), int(tokens[1])
        arr = [int(x) for x in tokens[2:2+n]]
        print(min_max_subarray_sum(arr, k))

```

Problem 7: Longest Subarray with Equal '0's and '1's

Problem Statement

Given a binary string S , find the length of the longest contiguous substring containing an equal number of '0's and '1's.

Constraints

- $1 \leq |S| \leq 10^5$

Edge Cases Handled

- No valid substring exists (returns 0).

Python Solution

```

def find_max_length_equal_01(s: str) -> int:
    prefix_map = {0: -1}
    curr_sum, max_len = 0, 0

    for i, char in enumerate(s):
        curr_sum += 1 if char == '1' else -1
        if curr_sum in prefix_map:
            max_len = max(max_len, i - prefix_map[curr_sum])
        else:
            prefix_map[curr_sum] = i

    return max_len

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        print(find_max_length_equal_01(tokens[0]))

```

Problem 8: Next Greater Element

Problem Statement

For every element in an array, print the first element to its right that is strictly greater than it. Output -1 if no such element exists.

Constraints

- $1 \leq N \leq 10^5$

Edge Cases Handled

- Monotonically decreasing arrays.

Python Solution

```
def next_greater_element(arr):
    n = len(arr)
    result = [-1] * n
    stack = []

    for i in range(n - 1, -1, -1):
        while stack and stack[-1] <= arr[i]:
            stack.pop()
        if stack:
            result[i] = stack[-1]
        stack.append(arr[i])

    return result

if __name__ == "__main__":
    import sys
    tokens = sys.stdin.read().split()
    if tokens:
        n = int(tokens[0])
        arr = [int(x) for x in tokens[1:n+1]]
        print(*(next_greater_element(arr)))
```